

# TrackVision

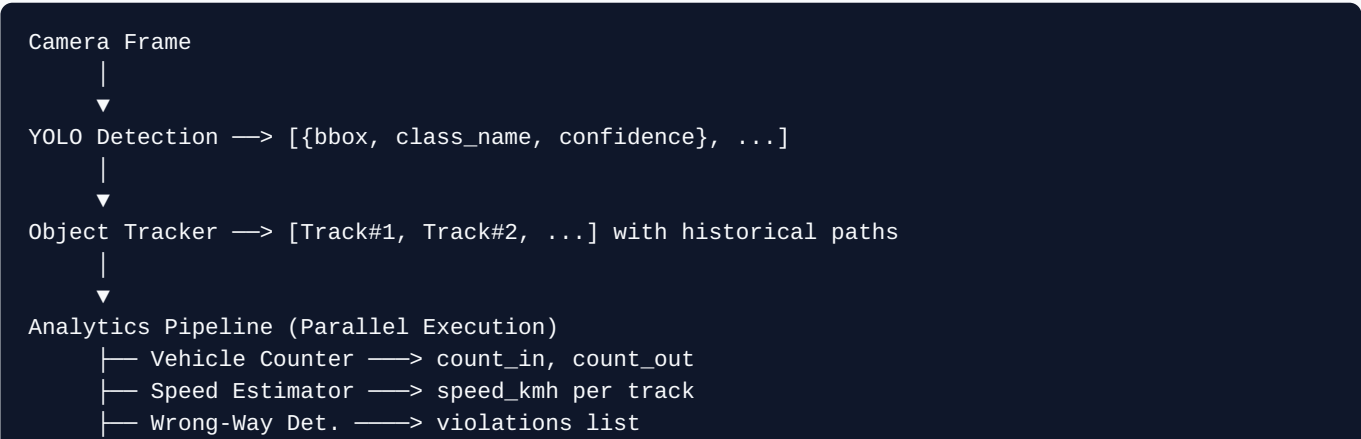
## 1. Overview

**TrackVision** is a complete, professional AI-powered vehicle tracking system built with Python and Flask. It captures video from any camera source, detects and tracks vehicles in real-time using deep learning, runs 6 different analytics modules simultaneously, makes local decisions (alerts, snapshots), and presents everything through a modern real-time web dashboard.

| FEATURE              | DESCRIPTION                                               |
|----------------------|-----------------------------------------------------------|
| AI Detection         | YOLOv8n identifies cars, trucks, buses, motorcycles       |
| Object Tracking      | Norfair maintains unique vehicle IDs across frames        |
| Vehicle Counting     | Counts vehicles crossing a configurable line              |
| Speed Estimation     | Calculates speed using pixel-to-meter calibration         |
| Wrong-Way Detection  | Flags vehicles moving in the wrong direction              |
| Intrusion Detection  | Detects vehicles entering forbidden zones                 |
| Congestion Analysis  | Monitors traffic density in real-time                     |
| Parking Monitoring   | Tracks occupancy of defined parking spots                 |
| Real-Time Streaming  | MJPEG stream to browser via WebSocket                     |
| Vehicle Detail Pages | Per-vehicle timeline, snapshot, speed chart               |
| Alert Management     | Categorized alerts with acknowledge system                |
| Flexible Camera      | Webcam, IP camera, RTSP, video files — switchable from UI |

## 2. System Architecture & Data Flow

The processing engine processes input video streams frame-by-frame and pushes telemetry asynchronously to web clients:



```

├─ Intrusion Det. —> intrusions list
├─ Congestion Ana. —> level, is_congested
├─ Parking Monitor —> occupancy dict
└─
  │
  ▼
Decision Engine (Log DB, Save Snapshot, Trigger Alerts)
  │
  ▼
Draw Overlays —> Frame Queue —> MJPEG Stream —> Browser
  │
  ▼
Socket.IO Emit (every 5 frames) —> Real-time UI Sync

```

## 3. Analytics Modules & Formulas

### Speed Estimation

Calculates frame displacement and maps pixel distances to physical velocity metrics using exponential moving averages:

$$speed\_kmh = (pixel\_distance / pixels\_per\_meter) / time\_seconds \times 3.6$$

$$smoothed\_speed = 0.4 \times speed\_kmh + 0.6 \times previous\_speed$$

### Wrong-Way Detection

Evaluates directional vectors against standard driving orientations to prevent traffic violations. Only triggers when speed exceeds 10 km/h to avoid false positives on stationary vehicles.

### Intrusion & Polygon Area Detection

Implements ray-casting point-in-polygon evaluations to flag restricted vehicular entry instantly upon crossing zone borders.

## 4. Database Schema (SQLite)

```

-- Vehicle detection logs
CREATE TABLE vehicle_logs (
  id INTEGER PRIMARY KEY AUTOINCREMENT,
  track_id INTEGER NOT NULL,
  vehicle_type TEXT NOT NULL,
  speed REAL DEFAULT 0,
  direction TEXT,
  timestamp TEXT NOT NULL,
  snapshot_path TEXT
);

-- Alert records
CREATE TABLE alerts (
  id INTEGER PRIMARY KEY AUTOINCREMENT,
  alert_type TEXT NOT NULL,      -- 'speed_violation', 'wrong_way', 'intrusion', 'congestion'
  severity TEXT NOT NULL,       -- 'critical', 'warning', 'info'
  message TEXT NOT NULL,

```

```

timestamp TEXT NOT NULL,
acknowledged INTEGER DEFAULT 0,
snapshot_path TEXT
);

-- Hourly aggregated counts for charts
CREATE TABLE hourly_counts (
  id INTEGER PRIMARY KEY AUTOINCREMENT,
  date TEXT NOT NULL,          -- '2026-08-04'
  hour INTEGER NOT NULL,       -- 0-23
  count INTEGER DEFAULT 0,
  UNIQUE(date, hour)
);

```

## 5. API Endpoints Reference

| METHOD | ENDPOINT                 | DESCRIPTION                                               |
|--------|--------------------------|-----------------------------------------------------------|
| GET    | /api/status              | Running state, FPS, source, active vehicles, alerts count |
| POST   | /api/camera/start        | Start camera with specific source payload                 |
| POST   | /api/camera/stop         | Halt video capture stream processing                      |
| GET    | /api/vehicles            | Paginated vehicle logs with date and type filtering       |
| GET    | /api/vehicles/<track_id> | Full tracking history and snapshot details for vehicle    |
| GET    | /api/analytics/summary   | Daily counts, violation stats, unacknowledged alerts      |
| POST   | /api/alerts/<id>/ack     | Acknowledge designated alert record                       |
| POST   | /api/clear-data          | Clear database tables and erase saved snapshots           |

## 6. System Configuration Reference

| SETTING                   | DEFAULT   | DESCRIPTION                                         |
|---------------------------|-----------|-----------------------------------------------------|
| CAMERA_SOURCE             | 0         | Default webcam index or stream path                 |
| YOLO_CONFIDENCE_THRESHOLD | 0.45      | Detection confidence threshold                      |
| COUNTING_LINE_Y           | 300       | Y-coordinate line location for vehicular counter    |
| PIXELS_PER_METER          | 15.0      | Distance conversion calibration parameter           |
| SPEED_LIMIT_KMH           | 60.0      | Velocity threshold for generating speed alerts      |
| TRACKER_TYPE              | "norfair" | Tracking algorithm engine ("norfair" or "centroid") |